

Smart Agriculture Crop Prediction Model

1. Importing Libraries

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, f1_score

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from xgboost import XGBClassifier

import matplotlib.pyplot as plt
import seaborn as sns

RANDOM_STATE = 54650
np.random.seed(RANDOM_STATE)
```

2. Dataset Loading

```
df = pd.read_csv("../Crop_Yield_Fertilizer.csv")

print(df.shape)
df.head()

(25000, 10)

{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"N","rawType":"float64","type":"float"},
{"name":"P","rawType":"float64","type":"float"},
{"name":"K","rawType":"float64","type":"float"},
{"name":"temperature","rawType":"float64","type":"float"},
{"name":"humidity","rawType":"float64","type":"float"},
{"name":"ph","rawType":"float64","type":"float"},
{"name":"rainfall","rawType":"float64","type":"float"},
{"name":"label","rawType":"object","type":"string"},
{"name":"yield","rawType":"float64","type":"float"},
{"name":"fertilizer","rawType":"object","type":"string"}],"ref":"c4bb7
827-4fb3-4780-b505-c31a9f9a40a6","rows":
[["0","90.0","42.0","43.0","20.87974371","82.00274423","6.502985292000
001","202.9355362","rice","71.19942820627362","DAP"],
["1","85.0","58.0","41.0","21.77046169","80.31964408","7.038096361","2
```

```
26.6555374","rice","81.62019870524915","DAP"],
["2","60.0","55.0","44.0","23.00445915","82.3207629","7.840207144","26
3.9642476","rice","80.47312983471406","Gypsum"],
["3","74.0","35.0","40.0","26.49109635","80.15836264","6.980400905","2
42.8640342","rice","75.17819600672037","DAP"],
["4","78.0","42.0","42.0","20.13017482","81.60487287","7.628472891","2
62.7173405","rice","75.48556318207437","Gypsum"]],"shape":
{"columns":10,"rows":5}}
```

3. Sanity Mandate Checks

```
# Missing values
print(df.isnull().sum())

# Crop label distribution
print(df['label'].value_counts())
```

```
N          0
P          0
K          0
temperature 0
humidity    0
ph          0
rainfall    0
label       0
yield       0
fertilizer  0
dtype: int64
label
orange      1196
kidneybeans 1180
papaya      1174
maize       1168
mungbean    1164
banana      1164
chickpea    1159
rice        1149
mothbeans   1149
coconut     1145
blackgram   1145
coffee     1140
watermelon  1133
jute        1129
pigeonpeas  1122
grapes      1111
apple       1110
cotton      1108
muskmelon   1098
pomegranate 1092
lentil      1085
```

```
mango          1079
Name: count, dtype: int64
```

4. Feature / Target Selection

```
FEATURES = [
    'N', 'P', 'K',
    'temperature', 'humidity', 'ph', 'rainfall'
]

TARGET = 'label'

X = df[FEATURES]
y = df[TARGET]
# including fertilizer and yeild would result in data leakage
```

5. Encode Target Labels

```
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

num_classes = len(label_encoder.classes_)
print("Number of crop classes:", num_classes)

Number of crop classes: 22
```

6. Preprocessing Pipeline

```
numeric_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, FEATURES)
    ]
)
```

7. Model Selection

```
# This model is capable of non-linear nutrient interactions
# and it robust to agricultural noises
model = XGBClassifier(
    n_estimators=500,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.9,
    colsample_bytree=0.9,
    objective='multi:softprob',
    num_class=num_classes,
```



```

max_bin=None, max_cat_threshold=None,
max_cat_to_onehot=None,
max_delta_step=None,
max_depth=5, max_leaves=None,
min_child_weight=None, missing=nan,
monotone_constraints=None,
multi_strategy=None,
n_estimators=500, n_jobs=None,
num_class=22, ...)))]

```

11. Evaluation

```

y_pred = pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=label_encoder.classes_
))

```

Accuracy: 0.9453333333333334

Classification Report:

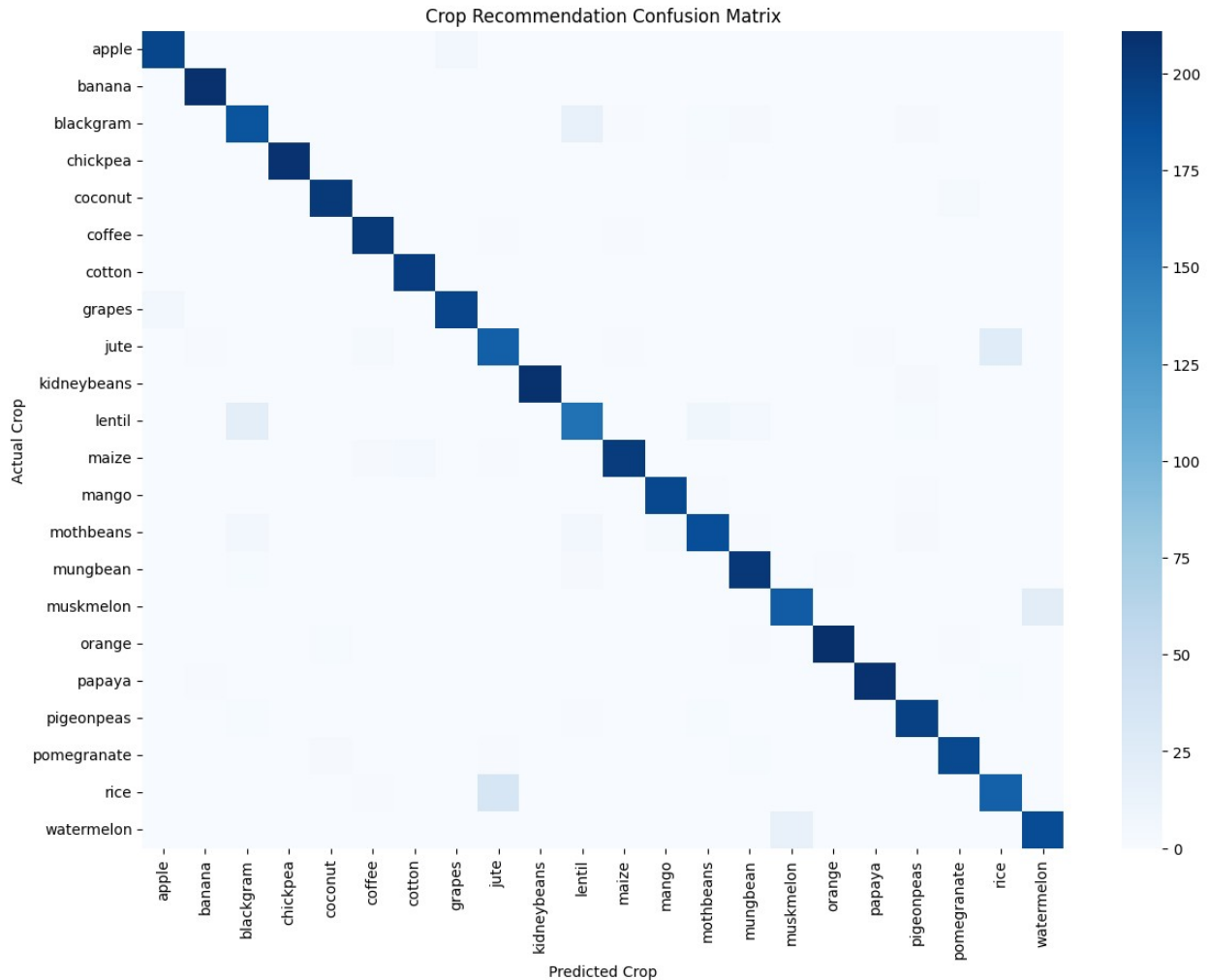
	precision	recall	f1-score	support
apple	0.97	0.96	0.97	200
banana	0.99	1.00	1.00	210
blackgram	0.85	0.88	0.87	206
chickpea	1.00	1.00	1.00	209
coconut	0.98	0.98	0.98	206
coffee	0.96	0.99	0.98	205
cotton	0.98	1.00	0.99	199
grapes	0.97	0.97	0.97	200
jute	0.82	0.85	0.83	203
kidneybeans	1.00	0.99	0.99	212
lentil	0.86	0.81	0.83	195
maize	0.99	0.96	0.97	210
mango	0.98	0.99	0.98	194
mothbeans	0.93	0.90	0.91	207
mungbean	0.94	0.97	0.96	210
muskmelon	0.92	0.88	0.90	198
orange	1.00	0.98	0.99	215
papaya	1.00	0.99	0.99	211
pigeonpeas	0.94	0.98	0.96	202
pomegranate	0.97	0.97	0.97	197
rice	0.87	0.83	0.85	207
watermelon	0.89	0.92	0.91	204

accuracy			0.95	4500
macro avg	0.94	0.94	0.94	4500
weighted avg	0.95	0.95	0.95	4500

12. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(14, 10))
sns.heatmap(
    cm,
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_,
    cbar=True
)
plt.xlabel("Predicted Crop")
plt.ylabel("Actual Crop")
plt.title("Crop Recommendation Confusion Matrix")
plt.show()
```



13. Cross-Validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)
```

```
macro_f1 = cross_val_score(
    pipeline,
    X,
    y_encoded,
    cv=cv,
    scoring='f1_macro',
    n_jobs=-1
)
```

```
print("Macro F1 scores:", macro_f1)
print("Mean Macro F1:", macro_f1.mean())
```

```
Macro F1 scores: [0.94275737 0.93925854 0.93902232 0.93789233
0.94160164]
```

```
Mean Macro F1: 0.9401064416460472
```

14. Inference Function

```
def recommend_crop(input_dict):  
    """  
    input_dict example:  
    {  
        'N': 90,  
        'P': 42,  
        'K': 43,  
        'temperature': 26,  
        'humidity': 80,  
        'ph': 6.5,  
        'rainfall': 200  
    }  
    """  
    input_df = pd.DataFrame([input_dict])  
    pred_encoded = pipeline.predict(input_df)[0]  
    return label_encoder.inverse_transform([pred_encoded])[0]
```

Example Prediction

```
sample_input = {  
    'N': 10,  
    'P': 10,  
    'K': 43,  
    'temperature': 10,  
    'humidity': 89,  
    'ph': 6.5,  
    'rainfall': 45  
}  
  
print("Recommended Crop:", recommend_crop(sample_input))  
Recommended Crop: pomegranate
```